



WEB TECHNOLOGIES

jQuery – Callback and Promises

Vinay Joshi

Department of
Computer Science and Engineering

- As seen in `setInterval`, `setTimeout` and `addEventListener`, a function accepts a function reference as an argument
- They will be called asynchronously based on timer or other events
- These function references are called **Callback functions**
- Example:
`div.addEventListener("keypress", function(){ ... });`

jQuery – Callbacks

Introduction



Synchronous code runs in sequence. This means that each operation must wait for the previous one to complete before executing.

```
console.log('One');  
console.log('Two');  
console.log('Three');  
// LOGS: 'One', 'Two', 'Three'
```

Asynchronous code runs in parallel. This means that an operation can occur while another one is still being processed.

```
console.log('One');  
    setTimeout(function(){  
        console.log('Two')  
    }, 1000);  
    console.log('Three');  
// Logs One Three two
```

Callback Hell

```
firstFunction(args, function() {  
    secondFunction(args, function() {  
        thirdFunction(args, function() {  
            // And so on...  
        });  
    });  
});
```

```
$(document).ready(function() {  
    ("li.g1").click(function() {  
        $(this).fadeOut(1000, function() {  
            $(this).fadeIn(2000, function() {  
                $(this).click()  
            });  
        });  
    });  
});
```

Difficulties to read and maintain.

If there is an error in one function, all the other functions would also get affected, making error handling more complicated.

"I Promise a Result!"

"Producing code" is code that can take some time

"Consuming code" is code that must wait for the result

A Promise is a JavaScript object that links producing code and consuming code

A JavaScript Promise object contains both the producing code and calls to the consuming code:

- Promise is a JavaScript object with a value that may not be available at the moment when the code line executes.
- These values are resolved at some point in the future.
- It allows you to write asynchronous code more synchronously.

jQuery – Promises

Introduction: Syntax for Callback and promises.



```
let myPromise = new Promise(function(myResolve, myReject) {  
  // "Producing Code" (May take some time)  
  
  myResolve(); // when successful  
  myReject();  // when error  
});  
  
// "Consuming Code" (Must wait for a fulfilled Promise)  
myPromise.then(  
  function(value) { /* code if successful */ },  
  function(error) { /* code if some error */ }  
);
```

A Promise Object Properties

JavaScript Promise object can be:

- Pending
- Fulfilled
- Rejected

The Promise object supports two properties: **state** and **result**.

While a Promise object is "pending" (working), the result is undefined.

When a Promise object is "fulfilled", the result is a value.

When a Promise object is "rejected", the result is an error object.

You cannot access the Promise properties state and result.

You must use a Promise method to handle promises.


```
// Promise constructor
let promise = new Promise(function(resolve, reject) {
    const x = "apple";
    const y = "apple"

    if (x === y) {
        resolve();
    } else {
        reject();
    }
});

// Consuming the Promise
promise
    .then(function () {
        console.log('Successful');
    })
    .catch(function () {
        console.log('Some error has occurred');
    });
```

- Promise constructor takes only one argument which is a callback function (and that callback function is also referred as an anonymous function too).
- Callback function takes two arguments, resolve and reject. Perform operations inside the callback function and if everything went well then call resolve.

- A promise is used to handle the asynchronous result of an operation.
- With Promises, we can defer execution of a code block until an async request is completed.
- The Promise object is created using the new keyword and contains the promise; this is an executor function which has a **resolve** and a **reject** callback
- Essentially, a promise is a returned object to which you attach callbacks, instead of passing callbacks into a function.

```
const promise = new Promise(function(resolve, reject) {  
  // promise description  
});
```

jQuery – Promises

Example



```
var weather;
const date = new Promise(
  function(resolve, reject) {
    weather = true; //usually a API call
    if (weather) {
      const dateDetails = {
        name: 'Cubana Restaurant',
        location: '55th Street',
        table: 5
      };
      resolve(dateDetails)
    } else {
      reject(new Error('Bad weather'))
    }
  }
);
```

```
date
  .then(function(done) {
    console.log('We are going on a date!')
    console.log(done)
  })
  .catch(function(error) {
    console.log(error.message)
  })
```

jQuery – Callback and Promises

Comparison



- Callbacks and Promises are not the same
- Callbacks are function passed to another function as a reference
- Chaining of Callbacks can be clumsy and lead to **Callback Hell**
- Promises use Callbacks and more elegant than Callbacks
- Chaining of Promises is supported



THANK YOU

Vinay Joshi

Department of Computer Science and Engineering

vinayj@pes.edu

+91 80 2672 6622



WEB TECHNOLOGIES

Single Page Applications & Asynchronous Communication

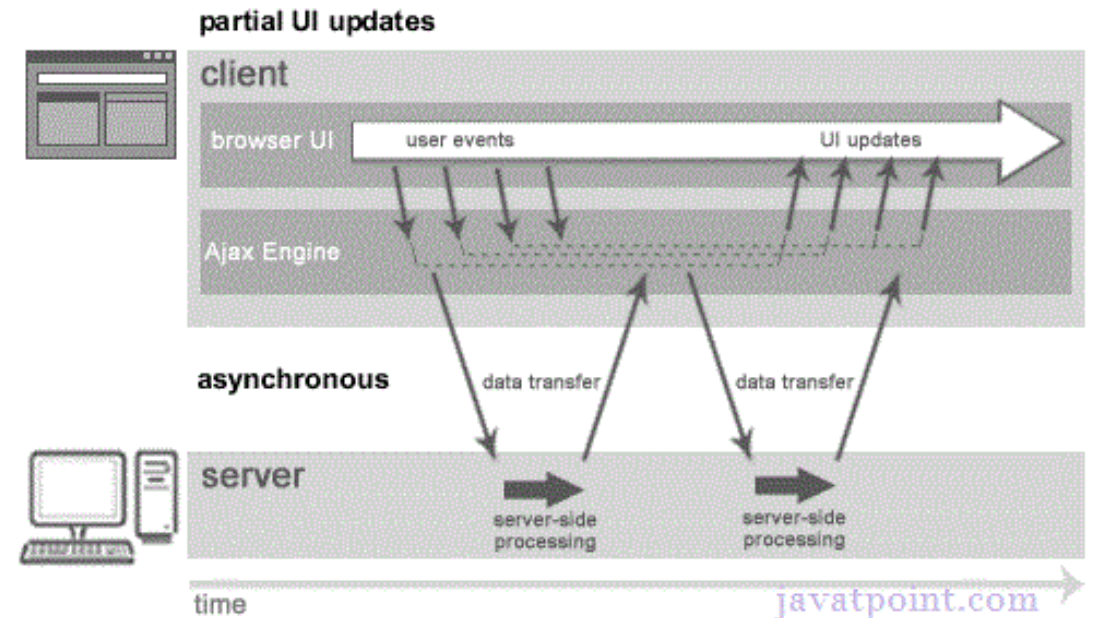
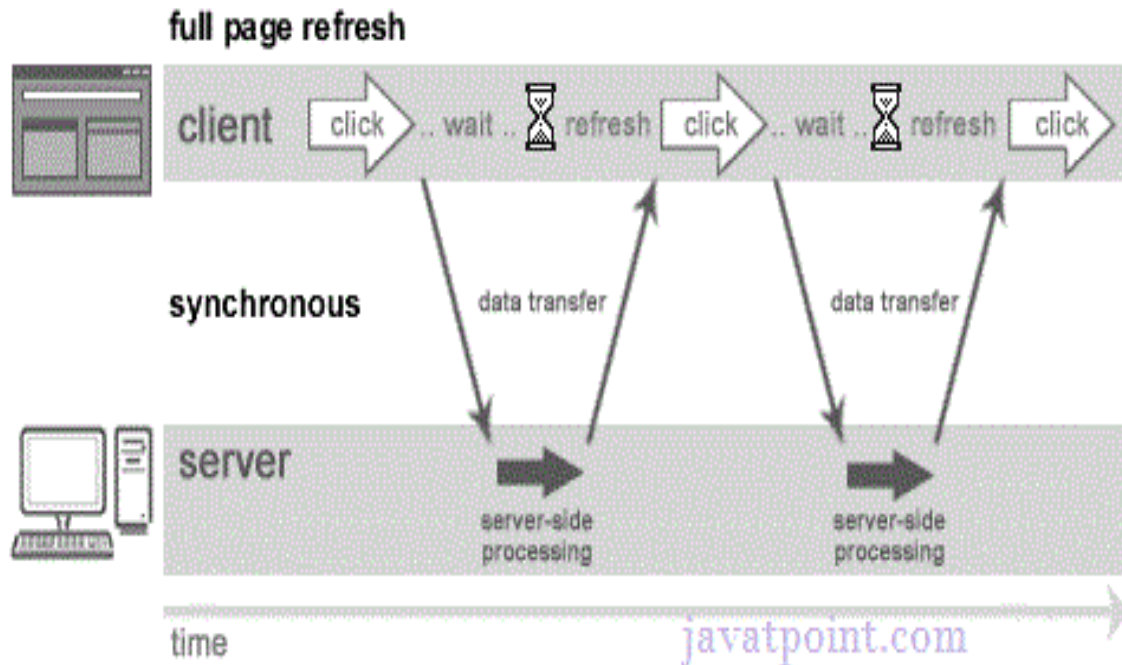
Vinay Joshi

Department of
Computer Science and Engineering

AJAX (Asynchronous JavaScript And XML)

A **synchronous** request blocks the client until operation completes i.e. browser is unresponsive. In such case, javascript engine of the browser is blocked.

An **asynchronous** request doesn't block the client i.e. browser is responsive. At that time, user can perform another operations also. In such case, javascript engine of the browser is not blocked.



- Instead of the default method of the browser loading entire new pages, a single-page application (SPA) interacts with the web browser by dynamically **rewriting the current web page with new data** from the web server
- Resources are dynamically loaded and added to the page as necessary, usually in response to user actions
- The page does not reload at any point in the process, nor does it transfer control to another page
- Can be built using
 - AJAX
 - Frameworks like ReactJS, AngularJS

Asynchronous Communication

AJAX(Asynchronous JavaScript And XML)



- Traditional web applications, upon request from user like clicking a link or submitting a form, a new page is loaded with requested resources
- Asynchronous applications, upon user action, updates a part of the page without reloading the entire page
- Approaches include
 - Setting src property of iFrame or img element
 - A more elegant and complete approach is use of XHR or XMLHttpRequest object
- First create an XHR object using
var xhr = new XMLHttpRequest();

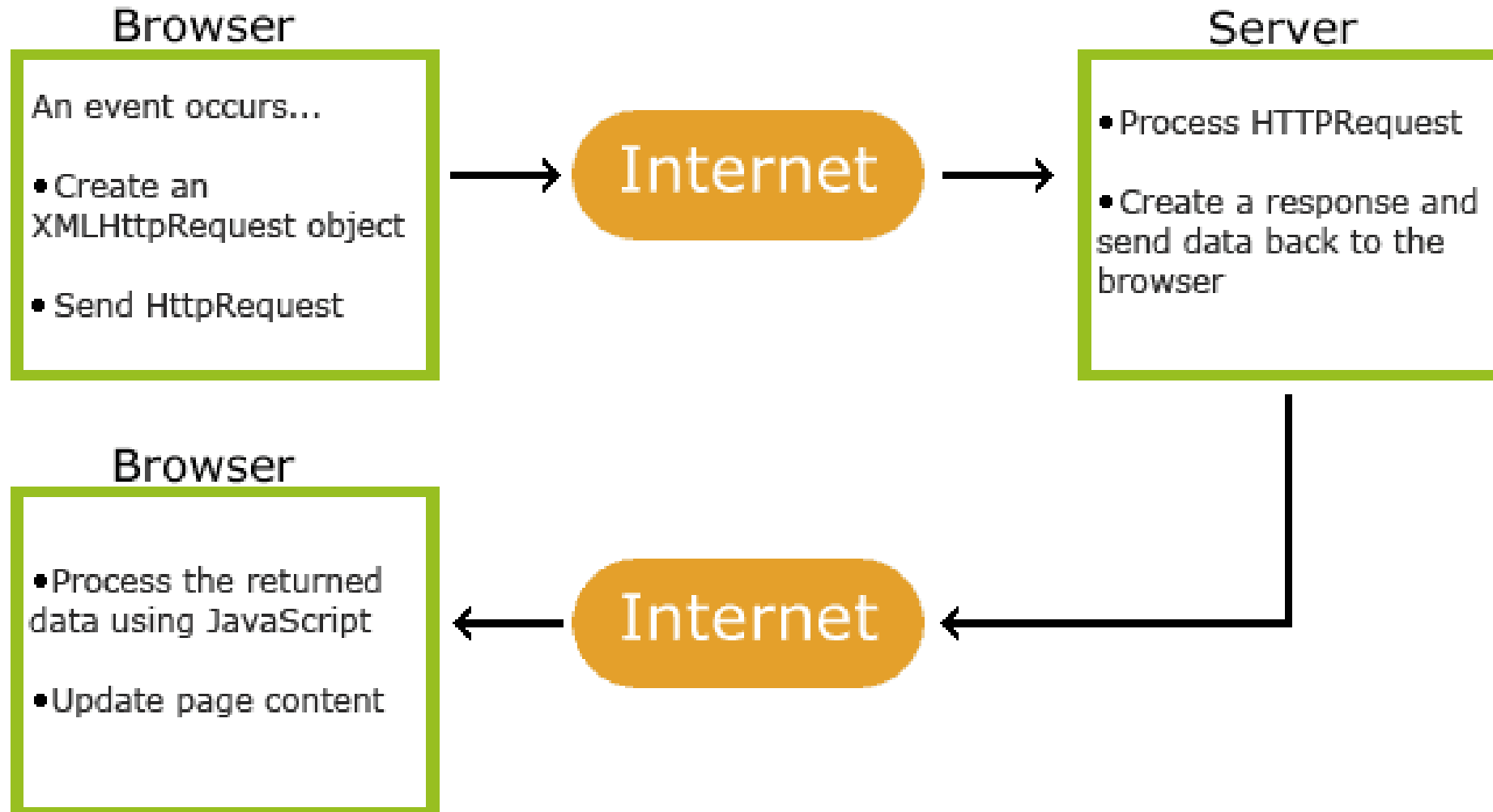
An object of XMLHttpRequest is used for asynchronous communication between client and server.

XMLHttpRequest object plays a important role.

1. User sends a request from the UI and a javascript call goes to XMLHttpRequest object.
2. HTTP Request is sent to the server by XMLHttpRequest object.
3. Server interacts with the database using JSP, PHP, Servlet, ASP.net etc.
4. Data is retrieved.
5. Server sends XML data or JSON data to the XMLHttpRequest callback function.
6. HTML and CSS data is displayed on the browser.

jQuery

AJAX- XHR



Asynchronous Communication

XHR object properties and methods

Properties / Methods	Description
<code>open(<i>method</i>, <i>url</i> [, <i>asynchronous</i>])</code>	Initializes the request in preparation for sending to the server. The <i>method</i> parameter is the HTTP method to use, for example “GET” or “POST”. The <i>url</i> is the relative or absolute URL the request will be sent to. The optional <i>asynchronous</i> parameter indicates whether <i>send()</i> returns immediately or after the request is complete (default is true, meaning it does not wait for response to come back)
<code>onreadystatechange</code>	Function to call whenever the readyState changes
<code>send([<i>body</i>])</code>	Initiates the request to the server. The <i>body</i> parameter should contain the body of the request, i.e., a string containing <i>fieldname=value&fieldname2=value2... for POSTs</i> or a null value for GET request

Asynchronous Communication

XHR object properties and methods

Properties / Methods	Description
readyState	Integer indicating the state of the request, either: 0 (uninitialized) 1 (loading) 2 (response headers received) 3 (some response body received) 4 (request complete)
status	HTTP status code returned by the server (e.g., 200, 404, etc.) 200: "OK" 403: "Forbidden" 404: "Page not found"
responseText	Full response from the server as a string (responseType property is set to "text" – default)
responseXML	A Document object representing the server's response parsed as an XML document (responseType property is set to "document")
response	Any other type of response received ((responseType property is set to "blob" or "json")

AJAX - Asynchronous Javascript And XML

XMLHttpRequest – Code Example



```
let xmlhttp = new XMLHttpRequest();
xmlhttp.open("GET", filepath, true);
xmlhttp.onreadystatechange=handler;
[xmlhttp.responseText="json"|"document"|"blob"] // default text
xmlhttp.send(null);

function handler() {
    if(this.readyState == 4 && this.status == 200) {
        // use this.response (json/blob) or this.responseText (text) or
        // this.responseXML (document) to update a part of the page
    }
}
```



THANK YOU

Vinay Joshi

Department of Computer Science and Engineering

vinayj@pes.edu

+91 80 2672 6622



WEB TECHNOLOGIES

XML vs. JSON

Vinay Joshi

Compiled by **Dr.Sarasvathi V**

Associate Professor

Department of Computer Science and Engineering

- XML stands for “extensible markup language”.
- XML was designed in 1996 and officially became a W3C standard in 1998.
- It was created to better represent data formats with a hierarchical structure.
- HTML
 - Display data and focus on how the data looks.
- XML
 - Structures, stores, and transfers information and describes what the data is.

- This data format fully supports hierarchical data structures and is very appropriate when receiving complex data as a response.
- It is also very human readable. Most browsers have built in XML readers that allow you to inspect XML files.
- Since XML was the first standard hierarchical data format, most APIs have built in functionality to automatically convert XML data streams into native data structures like objects.

- This data format is about three times as large as CSV. This is because each data element has an associated open and close parameter tag.

XML vs. JSON

JSON

- JSON stands for (Javascript Object Notation).
- It was invented in 2001 and became popularized by Yahoo and Google in 2005 and 2006.
- It was created as an alternative to XML. Like XML, however, it represents hierarchical data with the use of commas, curly braces and brackets.

XML Vs JSON HTML5

JSON PROS



- This data format supports hierarchical data while being smaller in size than XML.
- As its name implies, it was also created to more easily **parse data into native Javascript** objects, making it very useful for web applications.
- JSON is the best of both worlds with respect to CSV and XML. It's simple and compact like CSV, but supports hierarchical data like XML. Unlike XML, JSON formats are only about twice as large as CSV formats.

XML Vs JSON HTML5

JSON CONS



- This data format has a little bit less support than XML.
- Since JSON is relatively newer than XML, fewer APIs exist to automatically convert JSON to native data structures.
- However, this is rapidly changing because newer APIs and plugins are supporting both XML and JSON.

XML

```
<empinfo>
  <employees>
    <employee>
      <name>James Kirk</name>
      <age>40</age>
    </employee>
    <employee>
      <name>Jean-Luc Picard</name>
      <age>45</age>
    </employee>
    <employee>
      <name>Wesley Crusher</name>
      <age>27</age>
    </employee>
  </employees>
</empinfo>
```

JSON

```
{ "empinfo" :
  {
    "employees" : [
      {
        "name" : "James Kirk",
        "age" : 40,
      },
      {
        "name" : "Jean-Luc Picard",
        "age" : 45,
      },
      {
        "name" : "Wesley Crusher",
        "age" : 27,
      }
    ]
  }
}
```

XML vs. JSON

Differences

JSON	XML
Based on JavaScript language	Derived from SGML
Way of representing objects	Markup language to represent data items
No support for namespaces	Supports namespaces
Support arrays	Doesn't support arrays
Files are very easy to read as compared to XML	Documents are comparatively difficult to read and interpret
Doesn't use end tag	Has start and end tags
Less secure	More secured than JSON
Doesn't supports comments	Supports comments
Supports only UTF-8 encoding	Supports various encoding

XML Vs JSON HTML5

JSON

JSON String

```
'{"name":"John", "age":30, "car":null}'
```

XML Vs JSON HTML5

JSON

- JavaScript has a built in function for converting JSON strings into JavaScript objects:
`JSON.parse()`
- JavaScript also has a built in function for converting an object into a JSON string:
`JSON.stringify()`
- You can receive pure text from a server and use it as a JavaScript object.
- You can send a JavaScript object to a server in pure text format.
- You can work with data as JavaScript objects, with no complicated parsing and translations.

A text string is defined:

```
text = "<bookstore><book>" +  
"<title>Everyday Italian</title>" +  
"<author>Giada De Laurentiis</author>" +  
"<year>2005</year>" +  
"</book></bookstore>";
```

Parses a text string into an XML DOM object, and extracts the info from it with JavaScript:

An XML DOM parser is created:

```
parser = new DOMParser();
```

The parser creates a new XML DOM object using the text string:

```
xmlDoc = parser.parseFromString(text, "text/xml");
```

XML DOM object to XML string:

XMLSerializer interface provides `serializeToString()` method to construct an XML string

```
var oSerializer = new XMLSerializer();  
    var sXML = oSerializer.serializeToString(xmlDOM);  
    console.log(sXML);
```



THANK YOU

Vinay Joshi

Compiled by **Dr.Sarasvathi V**

Department of Computer Science and Engineering

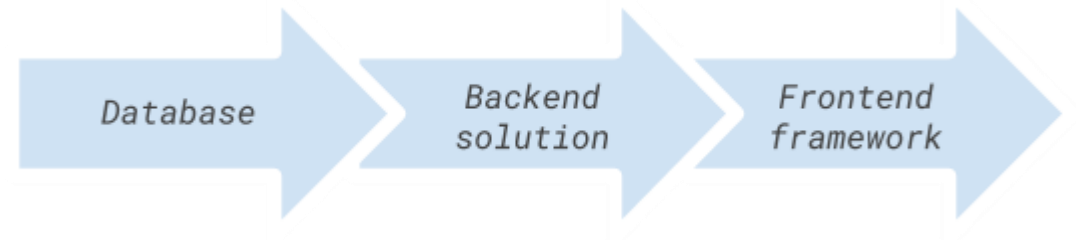


WEB TECHNOLOGIES

Web Development Stack

Prof. Vinay Joshi and Dr.Sarasvathi V
Department of Computer Science and Engineering

- A set of tools typically used in tandem to develop web apps.
- Refers to the technologies that that individual developer specializes in and use together to develop new pieces of software.
 - Front End web development stack
 - Back End web development stack
 - Full web development stack
 - Web technology sets that include all the essential parts of a modern app: the **frontend framework**, the **backend solution** and the **database (relational or document-oriented)**

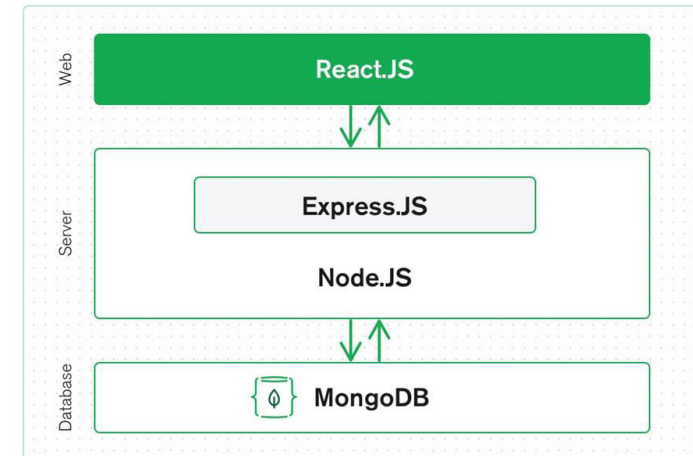


Web Development Stack

Full web development Stack

- As the web development world is continually changing, its technology stacks too changing
- Top web development stacks
 - MEAN
 - MERN
 - Meteor.js
 - Flutter
 - The serverless Technology stack
 - The LAMP technology stack
 - Ruby on Rails Tech Stack

- Stands for **M**ongoDB, **E**xpress, **R**eact, **N**ode
 - MongoDB - document database
 - Express(.js) - Node.js web framework
 - React(.js) - a client-side JavaScript library
 - Node(.js) - the premier JavaScript web server,
 - runtime environment



- Allows you to easily construct a **3-tier architecture** (frontend, backend, database) entirely using JavaScript and JSON MERN

Web Development Stack

React.JS

- Front End Library (The top tier of MERN Stack)
- The declarative JavaScript Library for creating dynamic client-side applications
- Build up complex interfaces through
 - simple Components,
 - connect them to data on your backend server, and
 - render them as HTML
- Strengths:
 - Handling stateful, data-driven interfaces
 - Great support for forms, error handling, events, lists, etc.



Web Development Stack

Node.JS and Express.JS

- Server side Framework (The middle tier of MERN Stack)
- Express.js server-side framework, running inside a Node.js server
- Has powerful models for URL routing (matching an incoming URL with a server function), and handling HTTP requests and responses
- In turn use Node.js MongoDB drivers, either via callbacks for using Promises, to access and update data in your MongoDB database



Web Development Stack

MongoDB

- Database Server (The bottom tier of MERN Stack)
- Application data store(user profiles, content, comments, uploads, events, etc.)
- JSON documents created in the front end can be stored directly in MongoDB for later retrieval through Node.JS and Express.JS



React.JS

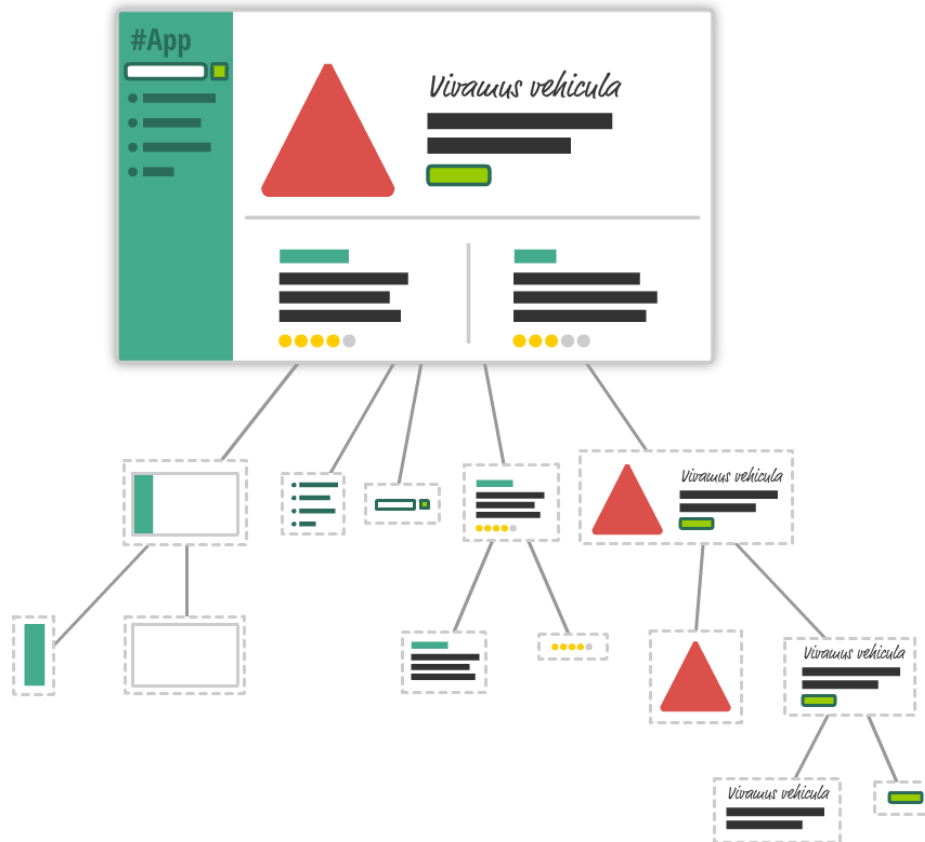
Introduction

- An open-source **JavaScript library** that is used for **building user interfaces** specifically for **single-page applications**.
- Used for handling the **view layer** for **web and mobile apps**.
- Allows developers to create **large web applications that can change data**, without reloading the page.
- Also allows to **create reusable UI components**.
- The main purpose of React is to **be fast, scalable, and simple**.
- **Works only on user interfaces** in the application.
- First created **by Jordan Walke**, a software engineer working for Facebook. First deployed on Facebook's newsfeed in 2011 and on Instagram.com in 2012.



React.JS

Introduction



React.JS

Why React?

- **Properties**

- Declarative, Simple, Component based, Supports server side, Mobile support, Extensive, Fast , Easy to learn

- **Single way data flow**

- A set of immutable values are passed to the components renderer as properties in its HTML tags. The component cannot directly modify any properties but can pass a call back function with the help of which we can do modifications.
- This complete process is known as “**properties flow down; actions flow up**”.

- **Virtual DOM**

- Creates an in-memory data structure cache which computes the changes made and then updates the browser.
- Allows a special feature that enables the programmer to code as if the whole page is rendered on each change whereas react library only renders components that actually change

- Two ways:
 - **Using node package manager(npm):** To setup a build environment for React that typically involved use of npm (node package manager), webpack, and Babel
 - **Directly importing Reactjs library in HTML Code.** Defined in two .js files (**React** and **ReactDOM**)
 - `<script src="https://unpkg.com/react@18/umd/react.development.js" crossorigin></script>`
 - `<script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js" crossorigin></script>`
 - The files differ for development and production.
 - When deploying, replace "development.js" with "production.min.js"



Creating React App Using node package manager(npm)

To create a new React app, you can use the **create-react-app** tool. Here's how you can do it step by step:

1. Download Node.js

Before you start, make sure Node.js is installed on your machine. Download and install it from [Node.js](https://nodejs.org/).

2. Install **create-react-app** (if you haven't already)

You need to globally install **create-react-app** using npm:

```
npm install -g create-react-app
```

Creating React App Using node package manager(npm)

3. Create a New React App

Once the `create-react-app` tool is installed, you can create a new React project by running:

```
npx create-react-app my-app-1
```

Replace `my-app-1` with your desired project name. This command will generate a new folder with all the necessary files and dependencies for your React project.

Creating React App Using node package manager(npm)

4. Navigate to the Project Directory

After the app is created, navigate into the project folder:

```
cd my-app-1
```

Creating React App Using node package manager(npm)

5. Start the Development Server

To start your React app, run the following command:

npm start

This will start the development server, and you should be able to see your React app running at <http://localhost:3000> in your web browser.

6. Open the Project in Your Code Editor

Open the project folder (**my-app-1**) in your preferred code editor (like VS Code) to start building your app.

Checking Installed Versions of React and React DOM

You can check the installed version of React using the terminal by running the following commands:

1. Check React Version

In your project directory, run the following command to check the version of **react**:

```
npm ls react
```

Checking Installed Versions of React and React DOM

2. Check React DOM Version

To check the version of `react-dom`, run:

`npm ls react-dom`

These commands will display the installed version of `react` and `react-dom`.

- A user control that has **code to represent visual interfaces and data**
- An isolated piece of code which can be reused in one or the other module
- Contains a root component in which other subcomponents are included
- 2 types of components in React.js
 - **Stateless Functional Component**
 - Includes simple JavaScript functions and immutable properties, i.e., the value for properties cannot be changed.
 - Use hooks to achieve functionality for making changes in properties using JS.
 - Used mainly for UI.
 - **Stateful Class Component**
 - Classes which extend the Component class from React library. The class component must include the render method which returns HTML.



- **Stateless Functional Component**

```
function Demo(props) {  
  return <h1> Welcome to REACT JS, {props.Name} </h1>;  
}
```

- **Stateful Class Component**

```
class HelloClass extends React.Component  
  {  
    render()  
    {  
      return React.createElement('h1', null, 'welcome to REACT JS');  
    }  
  }
```

With props

```
class Demo extends React.Component{  
  render(){  
    return <h1> Welcome to REACT JS, {this.props.Name} </h1>;  
  }  
}
```



- **Placeholder:** A place where the component will load
- **Create component:** Stateless or stateful
- **Call Component:** **ReactDOM.render()** is responsible for rendering a React component.
 - The first parameter is the component class name. Second parameter is the destination where the component is to be rendered. In our case, we render component inside the div id='root'.
 - Example: ReactDOM.render(
 React.createElement(HelloClass, null, null),
 document.getElementById('root')
);



- **React 17**

```
ReactDOM.render(  
  <h1>Batman</h1>,  
  document.querySelector("#container")  
);
```

The render method takes two arguments:

The HTML-like elements (aka JSX) you wish to output

The location in the DOM that React will render the JSX into

- **React 18**

Instead of ReactDOM.render , createRoot is used

```
const root = createRoot(container);  
root.render(element);
```

Example:

```
var destination =document.querySelector("#container");  
const root=ReactDOM.createRoot(destination);  
root.render(<h1>React World</h1>);
```

Create a React root for the supplied container and return the root.

The root can be used to render a React element into the DOM with render

- Uses a special syntax which allows you to **mix HTML with JavaScript**
- JSX is a JavaScript XML used in React applications. An extension to JavaScript.
- Uses HTML syntax to create elements and components.
- Has tag name, attributes, and children.
- JSX compiles the code into pure JavaScript which can be understood by the browser.
- Include the library: `<script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>`

```
<script type="text/babel">
```

```
  ReactDOM.render(<h1>Welcome to REACTJS</h1>,
```

```
    document.getElementById('root')
```

```
  );
```



React.JS

What is JSX?

```
<!DOCTYPE html>
<html>
<head>
  <title>React 18 App</title>
  <script src="https://unpkg.com/react@18/umd/react.development.js"></script>
  <script src="https://unpkg.com/react-dom@18/umd/react-
dom.development.js"></script>
  <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
</head>
<body>
  <div id="root"></div>
  <script type="text/babel">
    const root = ReactDOM.createRoot(document.getElementById('root'));
    root.render(<h1>Welcome to REACTJS</h1>);
  </script>
</body>
</html>
```



- A JavaScript compiler that can translate markup or programming languages into JavaScript.
- Available for different conversions.
- React uses Babel to convert JSX into JavaScript.
- Please note that `<script type="text/babel">` is needed for using Babel

```
<script type="text/babel">
```

```
    ReactDOM.render(<h1>Welcome to REACTJS</h1>,  
    document.getElementById('root')  
    );
```



- A JavaScript compiler that can translate markup or programming languages into JavaScript.
- Available for different conversions.
- React uses Babel to convert JSX into JavaScript.

```
import React from 'react';
```

```
import ReactDOM from 'react-dom/client';
```

```
const root = ReactDOM.createRoot(document.getElementById('root'));
```

```
root.render(<h1>Welcome to REACTJS</h1>);
```



JSX

```
ReactDOM.render(  
  <div>  
    <h1>Batman</h1>  
    <h1>Iron Man</h1>  
    <h1>Nicolas Cage</h1>  
    <h1>Mega Man</h1>  
  </div>,  
  destination );
```



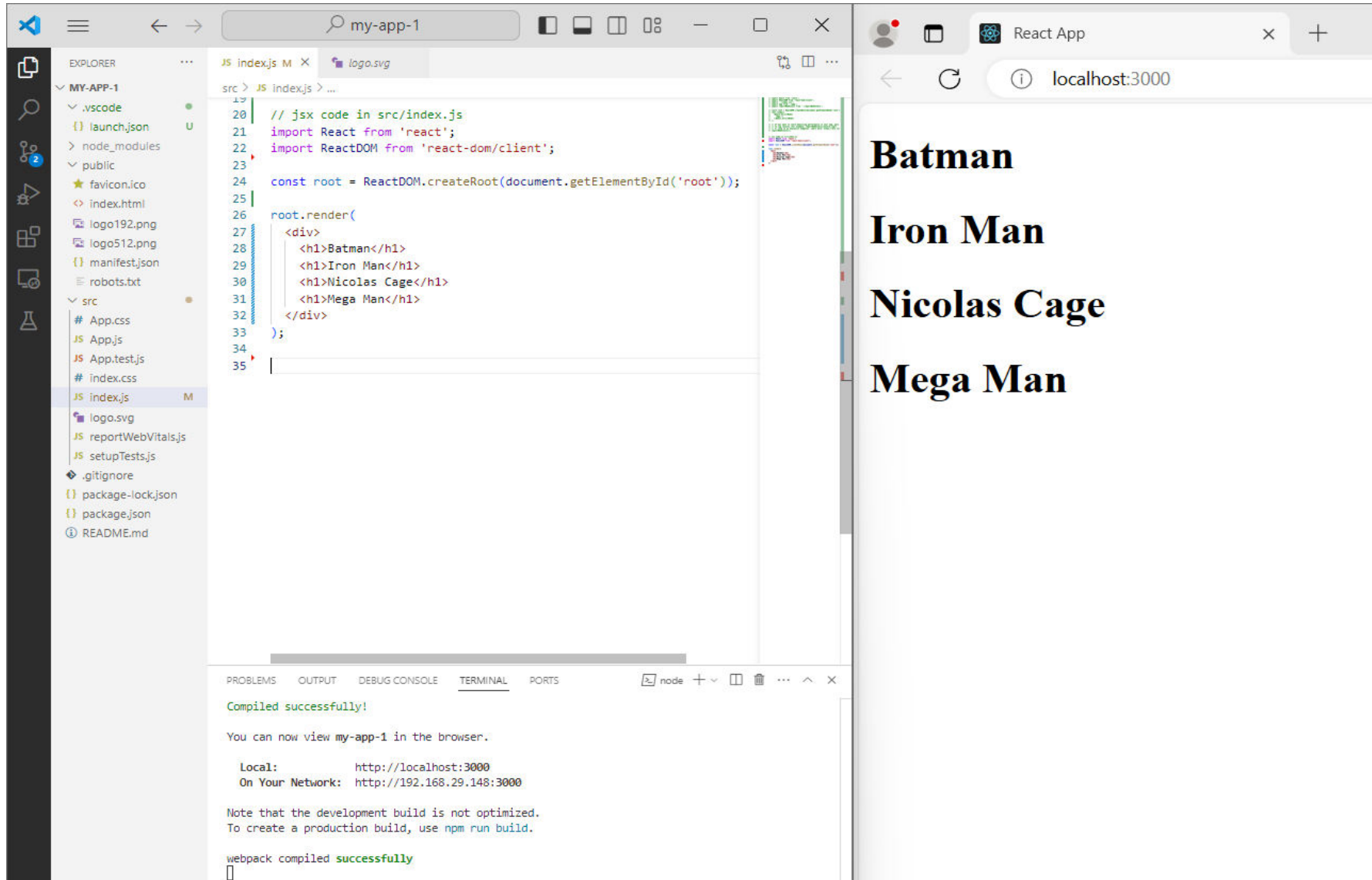
JavaScript

```
ReactDOM.render(  
  React.createElement ( "div", null,  
    React.createElement ( "h1", null, "Batman" ),  
    React.createElement ( "h1", null, "Iron Man" ),  
    React.createElement ( "h1", null, "Nicolas Cage" ),  
    React.createElement ( "h1", null, "Mega Man" ),  
  destination);
```



React.JS

Visuals Defined using JSX vs Pure JavaScript



The image shows a development environment with a code editor on the left and a browser on the right. The code editor displays the following JavaScript code in `src/index.js`:

```
20 // jsx code in src/index.js
21 import React from 'react';
22 import ReactDOM from 'react-dom/client';
23
24 const root = ReactDOM.createRoot(document.getElementById('root'));
25
26 root.render(
27   <div>
28     <h1>Batman</h1>
29     <h1>Iron Man</h1>
30     <h1>Nicolas Cage</h1>
31     <h1>Mega Man</h1>
32   </div>
33 );
34
35
```

The browser on the right shows the rendered output at `localhost:3000`:

Batman
Iron Man
Nicolas Cage
Mega Man

The terminal at the bottom of the code editor shows the following output:

```
Compiled successfully!

You can now view my-app-1 in the browser.

Local:      http://localhost:3000
On Your Network: http://192.168.29.148:3000

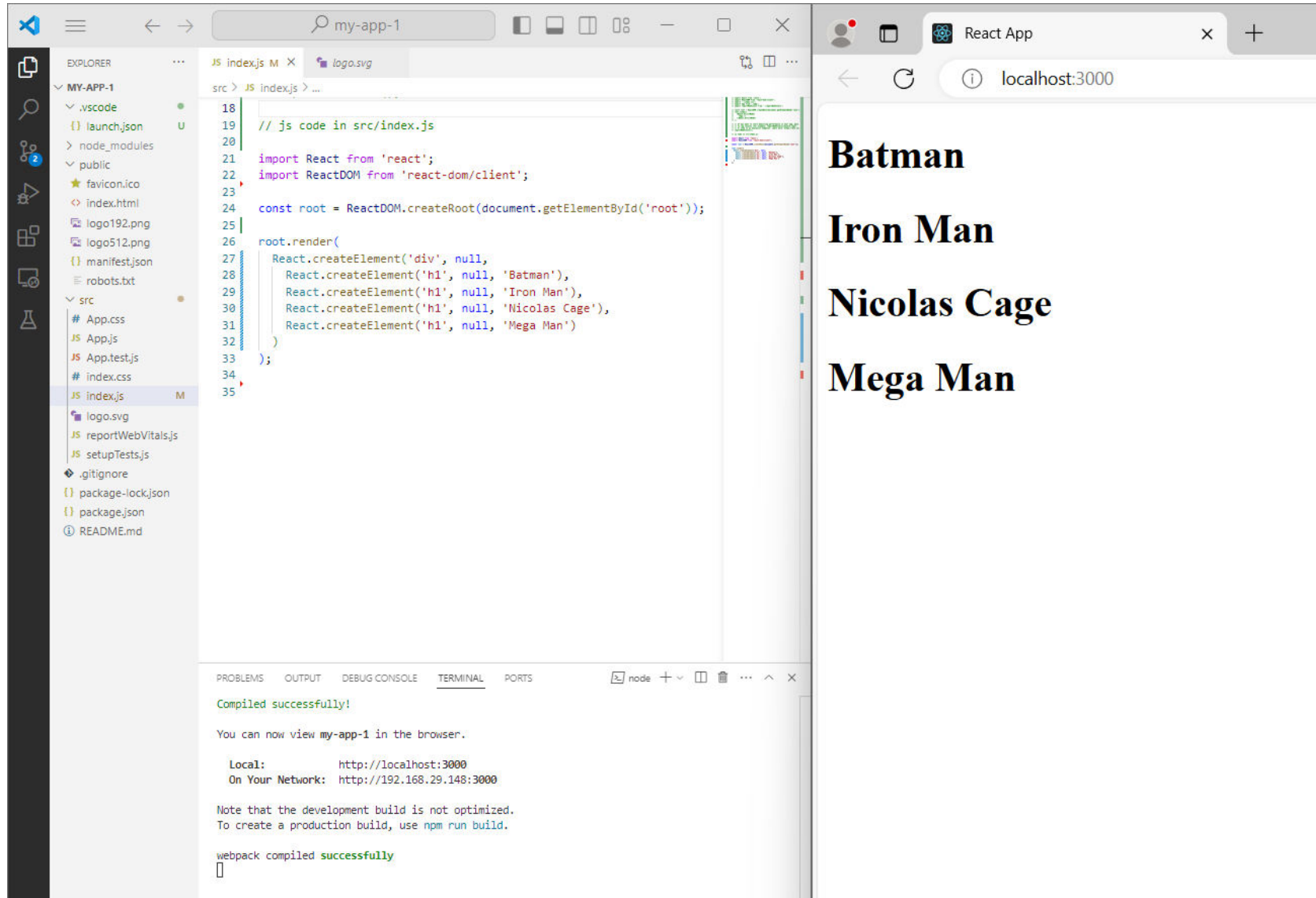
Note that the development build is not optimized.
To create a production build, use npm run build.

webpack compiled successfully
```



React.JS

Visuals Defined using JSX vs Pure JavaScript



The image shows a development environment with VS Code on the left and a web browser on the right. The browser displays the output of a React application running on localhost:3000, which lists four superhero names: Batman, Iron Man, Nicolas Cage, and Mega Man. The VS Code editor shows the source code for index.js, which uses React and ReactDOM to render these names. The terminal at the bottom of VS Code shows the successful compilation of the application.

```
// js code in src/index.js
import React from 'react';
import ReactDOM from 'react-dom/client';

const root = ReactDOM.createRoot(document.getElementById('root'));

root.render(
  React.createElement('div', null,
    React.createElement('h1', null, 'Batman'),
    React.createElement('h1', null, 'Iron Man'),
    React.createElement('h1', null, 'Nicolas Cage'),
    React.createElement('h1', null, 'Mega Man')
  )
);
```

Compiled successfully!

You can now view my-app-1 in the browser.

Local: http://localhost:3000
On Your Network: http://192.168.29.148:3000

Note that the development build is not optimized.
To create a production build, use `npm run build`.

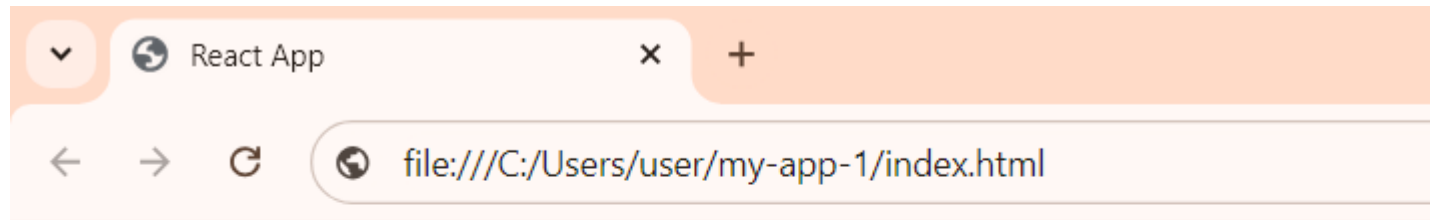
webpack compiled successfully



React.JS

Demonstration

- Using React, print welcome to REACTJS on the web page
- Use JSX and NonJSX both



Welcome to REACTJS



React.JS

Demonstration

Directly in HTML with React

1. JSX Approach

Create **index.html**:

```
<!DOCTYPE html>
<html>
<head>
  <title>React App</title>
  <script src="https://unpkg.com/react@18/umd/react.development.js" crossorigin></script>
  <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js" crossorigin></script>
  <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
</head>
<body>
  <div id="root"></div>
  <script type="text/babel">
    function App() {
      return <h1>Welcome to REACTJS</h1>;
    }

    const root = ReactDOM.createRoot(document.getElementById('root'));
    root.render(<App />);
  </script>
</body>
</html>
```



React.JS

Demonstration

Non-JSX Approach

Create `index.html`:

```
<!DOCTYPE html>
<html>
<head>
  <title>React App</title>
  <script src="https://unpkg.com/react@18/umd/react.development.js" crossorigin></script>
  <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js" crossorigin></script>
  <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
</head>
<body>
  <div id="root"></div>
  <script type="text/babel">
    function App() {
      return React.createElement('h1', null, 'Welcome to REACTJS');
    }

    const root = ReactDOM.createRoot(document.getElementById('root'));
    root.render(React.createElement(App));
  </script>
</body>
</html>
```





THANK YOU

Vinay Joshi and Dr.Sarasvathi V

Department of Computer Science and Engineering

vinayj@pes.edu

+91 8026726622

sarsvathiv@pes.edu



WEB TECHNOLOGIES

React JS – Components and Properties

Prof. Vinay Joshi and Prof. Sindhu R Pai
Department of Computer Science and Engineering

React.JS – Components and Properties

Agenda

- Introduction to Components
- Types of components
- First Stateful component
- Parameterized components
- Components with child element
- Usage of props.child
- Introduction to Properties
- Validating and setting property values
- Transferring properties

React.JS – Components and Properties

Introduction to Components

- Defines the **visuals and interactions** that make up what people see in app.

Need for Components

There may be elements which are similar

A need to make a change will result in changes in multiple places

Similar to functions, if we could write code related to the element in one place, changes can be minimized

Solution - Reusable piece of JavaScript code that output (via JSX) HTML elements



React.JS – Components and Properties

Types of Components

- **Stateless Functional Component**

- Includes simple JavaScript functions and immutable properties, i.e., the value for properties cannot be changed.
- Use hooks to achieve functionality for making changes in properties using JS.
- Used mainly for UI.

- **Stateful Class Component**

- Classes which extend the Component class from React library. The class component must include the render method which returns HTML.



React.JS – Components and Properties

First Stateful Component

Creation of a component:

```
class HelloWorld extends React.Component
{
    render()
    {
        return <p>Hello, componentized world!</p>;
    }
}
```

Calling a component: Add the JSX in the render method with a element with the tag name as the Component name

```
ReactDOM.render( <HelloWorld/>,
    document.querySelector("#container")
);
```



React.JS – Components and Properties

First Stateful Component

```
import React from 'react';
import ReactDOM from 'react-dom/client';

// Define the HelloWorld component
class HelloWorld extends React.Component {
  render() {
    return <p>Hello, componentized world!</p>;
  }
}

// Create a root for rendering
const root = ReactDOM.createRoot(document.querySelector("#container"));

// Render the HelloWorld component
root.render(<HelloWorld />);
```



React.JS – Components and Properties

Introduction to Properties(props)

- Properties are ways in which **React components can be customized**
- Properties to React components are same as what attributes are to HTML elements.
- **Props are like function arguments, you can send them into the component as attributes.**
- Their most basic use is in the form of attributes, in JSX.
- Are actually passed to a constructor and can be accessed by **this.props**
- Props are immutable
- camelCase in JSX

React.JS – Components and Properties

Parameterized components

Accepting the properties in the component and passing these properties while calling the components.

Creation of a component:

```
class HelloWorld extends React.Component {  
  render() { return <p>Hello, {this.props.greetTarget}!</p>; }  
}
```

We place it inside curly brackets - { }. In JSX, if you want something to get evaluated as an expression, you need to wrap that something inside curly brackets.

Calling a component:

```
ReactDOM.render(  
  <div>  
    <HelloWorld greetTarget="Batman"/> <HelloWorld greetTarget="Iron Man"/>  
  </div>,  
  document.querySelector("#container")  
);
```



React.JS – Components and Properties

Parameterized components

```
import React from 'react';
import ReactDOM from 'react-dom/client';

// Define the HelloWorld component
class HelloWorld extends React.Component {
  render() {
    return <p>Hello, {this.props.greetTarget}</p>;
  }
}

// Create a root for rendering
const root = ReactDOM.createRoot(document.querySelector("#container"));

// Render the HelloWorld components
root.render(
  <div>
    <HelloWorld greetTarget="Batman"/>
    <HelloWorld greetTarget="Iron Man"/>
  </div>
);
```



React.JS – Components and Properties

Usage of props.children

- Components can have one element, multiple elements, or none at all. It's value is respectively a single child node, an array of child nodes or undefined.
- **A component with children is always identified by an opening tag and a closing tag.**
- Each child must be placed between these two tags
- **Example**
- `<MyComponent>foo</MyComponent>`
 - MyComponent has one children foo
- A special property that is passed to components automatically
- **Content between those tags is passed as a special prop: props.children**



React.JS – Components and Properties

Components with Child elements

Your components can have children. The child elements in a component can be accessed by the **children property** accessed by **this.props.children**.

Creation of a component:

```
class Buttonify extends React.Component { render()  
  { return( <div>  
    <button type={this.props.behavior}>  
      {this.props.children}</button>  
    </div> );  
  }  
}
```

Calling a component:

```
ReactDOM.render(  
  <div>  
    <Buttonify behavior="submit">SEND DATA</Buttonify>  
  </div>, document.querySelector("#container")  );
```



React.JS – Components and Properties

Components with Child elements

```
import React from 'react';  
import ReactDOM from 'react-dom/client';
```

```
class Buttonify extends React.Component {  
  render() {  
    return (  
      <div>  
        <button type={this.props.behavior}>  
          {this.props.children}  
        </button>  
      </div>  
    );  
  }  
}
```

// Define the Buttonify component

```
const root = ReactDOM.createRoot(document.querySelector("#container"));  
root.render(  
  component  
    <div>  
      <Buttonify behavior="submit">SEND DATA</Buttonify>  
    </div>  
  );
```

// Create a root for rendering
// Render the Buttonify



React.JS – Components and Properties

Components with Child elements

Ensure your HTML file includes the container element:

```
<!DOCTYPE html>
<html>
<head>
  <title>React 18 App</title>
  <script src="https://unpkg.com/react@18/umd/react.development.js" crossorigin></script>
  <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js" crossorigin></script>
  <script type="module" src="path-to-your-js-file.js"></script> <!-- Make sure to reference your
JS file here -->
</head>
<body>
  <div id="container"></div>
</body>
</html>
```



React.JS – Components and Properties

Validating and Setting the property values

- Validating the type of properties
 - Possible by providing any of the types in **React.PropTypes.***
- Setting the default property values
 - Can be overridden by properties passed in initialisation using **getDefaultProps()** method
 - Must return an object. The contents of that object are up to us
- Coding examples

React.JS – Components and Properties

Validating and Setting the property values

```
class MyComponent extends React.Component {  
  render() {  
    return <div>{this.props.message}</div>;  
  }  
}  
  
MyComponent.defaultProps = {  
  message: 'Default message'  
};
```

Notes

1. **Deprecated Method:** The **getDefaultProps** method was used in older versions of React but is deprecated in favor of **defaultProps**. It is not recommended to use **getDefaultProps** in new code.
2. **PropTypes:** Validating props with **PropTypes** is supported in both React 17 and React 18.

React.JS – Components and Properties

Validating and Setting the property values

//Greeting.js

```
import React from 'react';
```

```
// Define the component
class Greeting extends
React.Component {
  render() {
    return <h1>Hello,
{this.props.name}</h1>;
  }
}
```

```
// Set default props
Greeting.defaultProps = {
  name: 'Guest'
};
```

```
export default Greeting;
```

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import Greeting from './Greeting'; // Assume the component is saved in Greeting.js

const root = ReactDOM.createRoot(document.querySelector("#container"));

root.render(
  <div>
    <Greeting name="Batman" /> {/* This will render: Hello, Batman! */}
    <Greeting /> {/* This will render: Hello, Guest! */}
  </div>
);
```

React.JS – Components and Properties

Transferring Properties

- Sending properties down the structural tree while working with multiple components in a hierarchy
- **transferPropsTo():**
 - Passes all the properties of a parent component to a child component unless we explicitly set the value of a child component's property
- Coding examples

React.JS – Components and Properties

Transferring Properties

In modern React (including React 17 and 18), the **transferPropsTo** method is outdated and has been replaced by a more standardized approach to passing props through components. **Instead of transferPropsTo, you should use explicit prop passing.**

//Parent Component

```
import React from 'react';  
import ChildComponent from './ChildComponent'; // Assume ChildComponent is in the same directory
```

```
class ParentComponent extends React.Component {  
  render() {  
    return (  
      <div>  
        <ChildComponent name="React" age={8} />  
      </div>  
    );  
  }  
}
```

```
export default ParentComponent;
```


React.JS – Components and Properties

Transferring Properties

//Child Component

```
import React from 'react';
```

```
class ChildComponent extends React.Component {  
  render() {  
    return (  
      <div>  
        <h1>Name: {this.props.name}</h1>  
        <h2>Age: {this.props.age}</h2>  
      </div>  
    );  
  }  
}
```

```
export default ChildComponent;
```



THANK YOU

Vinay Joshi and Sindhu R Pai

Department of Computer Science and Engineering

vinayj@pes.edu

+91 9886703973

sindhurpai@pes.edu

+91 8277606459



WEB TECHNOLOGIES

React JS – Styling the Components and Complex Components

Prof. Vinay Joshi and Prof. Sindhu R Pai
Department of Computer Science and Engineering

React.JS – Styling the Components

Agenda

- Introduction
- Inline CSS
- Usage of props.children
- Demo code

React.JS – Styling the Components

Introduction

- Different ways to style React Components
 - **Inline CSS** - using style attribute
 - CSS in JS - Using Libraries JSS and Styled Components
 - CSS modules – css loader and Sass & SCSS
 - Stylable



React.JS – Styling the Components

Inline CSS

- To style an element with the inline **style attribute**, the value must be a JavaScript object
- In JSX, JavaScript expressions are written inside curly braces and JavaScript objects also use curly braces

- Example:

```
class MyHeader extends React.Component {  
  render() {    return (  
    <h1 style={{color: "red"}}>Hello Style!</h1>  
    <p>Add a little style!</p>    );  }  
}
```

- camelCased Property Names

```
<h1 style={{backgroundColor: "lightblue"}}>Hello Style!</h1>
```



React.JS – Styling the Components

Inline CSS

```
import React from 'react';

class MyHeader extends React.Component {
  render() {
    return (
      <div>
        <h1 style={{ color: 'red', backgroundColor: 'lightblue' }}>
          Hello Style!
        </h1>
        <p>Add a little style!</p>
      </div>
    );
  }
}

export default MyHeader;
```



React.JS – Styling the Components

Inline CSS continued..

- Creation of javascript object which contains the css properties and values in the form of key value pair
- Passing this as a value to style attribute
- Example:

```
class Letter extends React.Component {  
  render() {var letterStyle = {    padding: 10,        margin: 10,        backgroundColor: "#ffde00",  
                                color: "#333",        display: "inline-block",    fontFamily: "monospace",  
                                fontSize: 32,        textAlign: "center"  
    };  
  return(  
    <div style={letterStyle}>  
      {this.props.children}  
    </div>    ); }  
}
```



React.JS – Styling the Components

Inline CSS continued..

```
import React from 'react';

class Letter extends React.Component {
  render() {
    // Define the style object
    const letterStyle = {
      padding: 10,
      margin: 10,
      backgroundColor: "#ffde00",
      color: "#333",
      display: "inline-block",
      fontFamily: "monospace",
      fontSize: 32,
      textAlign: "center"
    };

    return (
      <div style={letterStyle}>
        {this.props.children}
      </div>
    );
  }
}

export default Letter;
```



React.JS – Styling the Components

Demo

- Create the following letters using react components
- Additionally, apply styles to them differently by passing styling properties



React.JS – Complex Components

Agenda

- Introduction
- Approach followed
- Complex Component creation
- Sub component creation
- Properties from parent to sub component
- Modified parent and sub components
- Transferring properties



THANK YOU

Vinay Joshi and Sindhu R Pai

Department of Computer Science and Engineering

vinayj@pes.edu

+91 80 2672 6622

sindhurpai@pes.edu

+91 8277606459



WEB TECHNOLOGIES

HTML 5 – Audio, Video and Progress elements

Prof.Vinay Joshi and Dr.Sarasvathi V

Department of Computer Science and Engineering

HTML5 - Audio

Audio elements

- A standard approach for browser to play audio without the use of plug-in.
- Mp3, Wav and ogg formats supported by HTML5

Syntax

```
<audio controls="c1">  
  <source src="song.ogg" type="audio/ogg" />  
  <source src="song.mp3" type="audio/mp3" />  
  Your browser does not support the audio element.  
</audio>
```

HTML5 - Audio

Audio element properties

Attribute	Value	Description
autoplay	autoplay	Specifies that the audio will start playing as soon as it is ready.
controls	controls	Specifies that controls will be displayed, such as a play button.
loop	loop	Specifies that the audio will start playing again (looping) when it reaches the end
preload	preload	Specifies that the audio will be loaded at page load, and ready to run. Ignored if autoplay is present.
src	<i>url</i>	Specifies the URL of the audio to play

- A standard approach for browser to play video without the use of plug-in

Syntax

```
<video width="320" height="240" controls="controls">  
  <source src="movie.ogg" type="video/ogg" />  
  <source src="movie.mp4" type="video/mp4" />  
  <source src="movie.webm" type="video/webm" />  
  Your browser does not support the video tag.  
</video>
```


HTML5 - Video

Video element properties

Attribute	Value	Description
audio	muted	Defining the default state of the the audio. Currently, only "muted" is allowed
autoplay	autoplay	If present, then the video will start playing as soon as it is ready
controls	controls	If present, controls will be displayed, such as a play button
height	<i>pixels</i>	Sets the height of the video player
loop	loop	If present, the video will start over again, every time it is finished
poster	<i>url</i>	Specifies the URL of an image representing the video
preload	preload	If present, the video will be loaded at page load, and ready to run. Ignored if "autoplay" is present
src	<i>url</i>	The URL of the video to play
width	<i>pixels</i>	Sets the width of the video player

- The <progress> tag represents the completion progress of a task.
- Always add the <label> tag for describing the task!
- Use JavaScript to manipulate the value of the progress bar

Syntax:

```
<label for="file">Downloading progress:</label>
```

```
<progress id="file" value="32" max="100"> 32% </progress>
```

Attribute	Value	Description
<u>max</u>	<i>number</i>	Specifies how much work the task requires in total. Default value is 1
<u>value</u>	<i>number</i>	Specifies how much of the task has been completed



THANK YOU

Vinay Joshi and Dr.Sarasvathi V

Department of Computer Science and Engineering



WEB TECHNOLOGIES

HTML 5 – Canvas & SVG

Prof.Vinay Joshi and Dr.Sarasvathi V
Department of
Computer Science and Engineering

HTML5 - Canvas

Canvas element

- Uses JavaScript to draw graphics on a web page
- A rectangular area, and you control every pixel of it

Syntax

```
<canvas id="myCanvas" width="200" height="100">  
    Canvas is not supported  
</canvas>
```

- The canvas element has no drawing abilities of its own. All drawing must be done inside a JavaScript using the context object

```
<script type="text/javascript">  
  var c=document.getElementById("#myCanvas");  
  var ctx=c.getContext("2d");  
  ctx.fillStyle="#FF0000";  
  ctx.fillRect(50,50,150,75);  
</script>
```



Method	Description
<code>fillRect(x, y, width, height)</code>	Draws a filled rectangle
<code>strokeRect(x, y, width, height)</code>	Draws a rectangular outline
<code>clearRect(x, y, width, height)</code>	Clears the specified rectangular area, making it fully transparent
<code>moveTo(x, y)</code>	Moves the pen to the coordinates specified by x and y
<code>lineTo(x, y)</code>	Draws a line from the current drawing position to the position specified by x and y
<code>arc(x, y, r, sAngle, eAngle, anticlockwise)</code>	Draws an arc centered at (x, y) with radius r starting at sAngle and ending at eAngle going anticlockwise (defaulting to clockwise).
<code>arcTo(x1, y1, x2, y2, radius)</code>	Draws an arc with the given control points and radius, connected to the previous point by a straight line

Method	Description
<code>createLinearGradient(x1, y1, x2, y2)</code>	Creates a linear gradient object with a starting point of (x1, y1) and an end point of (x2, y2).
<code>createRadialGradient(x1, y1, r1, x2, y2, r2)</code>	Creates a radial gradient. The parameters represent two circles, one with its center at (x1, y1) and a radius of r1, and the other with its center at (x2, y2) with a radius of r2.
<code>fillText(text, x, y [, maxWidth])</code>	Fills a given text at the given (x,y) position. Optionally with a maximum width to draw.
<code>strokeText(text, x, y [, maxWidth])</code>	Strokes a given text at the given (x,y) position. Optionally with a maximum width to draw.
<code>drawImage(image, x, y [,width, height])</code>	Draws the CanvasImageSource specified by the image parameter at the coordinates (x, y) with optional width and height

HTML5 – SVG

SVG Element

- SVG stands for Scalable Vector Graphics.
- SVG defines vector-based graphics using HTML elements
- SVG graphics do NOT lose any quality if they are zoomed or resized

```
<svg width="100" height="100">  
  <circle cx="50" cy="50" r="40" stroke="green" stroke-width="4"  
    fill="yellow" />  
</svg>
```



- `<rect width="300" height="100" style = "fill:rgb(0,0,255); stroke-width:3; stroke:rgb(0,0,0)" />`
- `<circle cx="50" cy="50" r="40" stroke="black" stroke-width="3" fill="red" />`
- `<polygon points="200 100,300 100,300 200,200 200" style = "fill:lime; stroke:purple; stroke-width:1" />`
- `<ellipse cx="200" cy="80" rx="20" ry="50" style = "fill:yellow; stroke:purple; stroke-width:2" />`
- `<text x="0" y="15" fill="red" transform="rotate(30 20,40)">I love SVG </text>`



THANK YOU

Prof. Vinay Joshi and Dr.Sarasvathi V
Department of Computer Science and Engineering



WEB TECHNOLOGIES

HTML 5 – Geo Location

Prof.Vinay Joshi and Dr.Sarasvathi V
Department of
Computer Science and Engineering

- Enables your web application to obtain the geographical position of your website visitors
- The user has to accept to share their location,
- The Geolocation API is work with the navigator.geolocation object.
- Accessed via JavaScript, through the **navigator.geolocation** object.

- navigator.geolocation object allows you to access geo location through two primary functions:
 - getCurrentPosition()
 - Returns the location of the visitor as a one-time snapshot
 - watchPosition()
 - returns the location of the visitor every time the location changes

”

Syntax : getCurrentPosition(success, error, options)

- Both functions take the following parameters:
 - Success callback function
 - Error callback function (optional)
 - Geo location options object (optional)
 - If the getCurrentPosition() method is successful, it returns a coordinates object to the function specified in the parameter (success)

HTML5 – Geo Location

Introduction

The Geolocation API uses three methods of Geolocation interface which are given following:

Methods	Description
<code>getCurrentPosition()</code>	It identifies the device or the user's current location and returns a position object with data.
<code>watchPosition()</code>	Return a value whenever the device location changes.
<code>clearWatch()</code>	It cancels the previous <code>watchPosition()</code> call

```
navigator.geolocation.getCurrentPosition(showPosition);  
function showPosition(position) {  
    x.innerHTML = "Latitude: " + position.coords.latitude +  
        "<br>Longitude: " + position.coords.longitude;  
}
```

```
navigator.geolocation.getCurrentPosition( function(position) {  
    console.log("your position is: " + position.coords.latitude + ", " +  
        position.coords.longitude);  
} );
```

* `getCurrentPosition` can be replaced with `watchPosition` if you need to receive updates to location



THANK YOU

Vinay Joshi and Dr.Sarasvathi V

Department of Computer Science and Engineering



WEB TECHNOLOGIES

jQuery – JavaScript Library

Vinay Joshi

Department of
Computer Science and Engineering

- Fast and concise
- Simplifies the interaction between HTML and JavaScript
- Lightweight : 19KB in size (Minified and Gzipped)
- CSS1 - 3 Complaint
- Cross Browser
 - (IE 6.0+, FF 2+, Safari 3.0+, Opera 9.0+, Chrome)
- It makes things like HTML document traversal and manipulation, event handling, animation, and Ajax

- Download jQuery from
http://docs.jquery.com/Downloading_jQuery

- Include the library in your web page

```
<head>
```

```
<script src="path/to/jquery-x.x.js"></script>
```

```
</head>
```

OR

- Include from a CDN (Content Delivery Network)

```
<head>
```

```
<script src = "https://code.jquery.com/jquery-3.5.1.min.js"> </script>
```

```
</head>
```

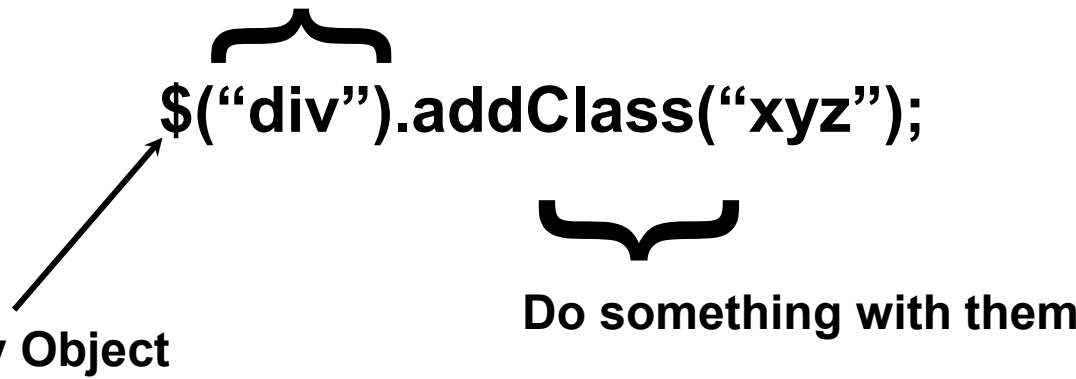
Vanilla JavaScript:

```
let paras = document.querySelectorAll("p")  
for (let i=0; i<paras.length; i++)  
    paras[i].style.color = "red"
```

jQuery:

```
$("p").css("color", "red");
```

Find Some Elements



```
$("div").addClass("xyz");
```

jQuery Object


Do something with them

Select By	Example
ID	<code>\$("#header")</code>
Class	<code>\$(".updated")</code>
Tag Name	<code>\$("table")</code>
Combination	<code>\$("table.user-list")</code> or <code>\$("#footer ul.menu li")</code>
Basic Filters	<code>:first</code> , <code>:last</code> , <code>:even</code> , <code>:odd</code>
Content Filters	<code>:empty</code> , <code>:contains(text)</code> , <code>:has(selector)</code>
Attribute Filters	<code>[attribute]</code> , <code>[attribute=value]</code> , <code>[attribute!=value]</code>
Forms	<code>:input</code> , <code>:text</code> , <code>:submit</code> , <code>:password</code> , <code>:enabled</code> , <code>:checked</code>

Action	Example
DOM Manipulation	before(), after(), append(), appendTo()
Attributes	addClass(), css(), attr(), html(), val(), text()
Events	click(), on(), bind(), unbind(), live()
Effects	hide(), fadeOut(), toggle(), animate()
AJAX	load(), get(), ajax(), post(), getJSON()

- `append()` – adds a set of elements to the end of the children
 - `$(“pelem”).append($c1[, c2, ...])`
 - **Eg: `$(“ul”).append(newli1,newli2)`**
`// $c1, $c2, ... will be appended to child elements of $pelem`
 - Similar methods : `appendTo`, `prepend`
- `after()` - adds a set of elements after the specified element
 - `$(“elem”).after($e1[, $e2,...])`
 - **Eg: `$(“li:last”).after(newli1)`**
`// $e1, $e2 will be added after $elem under the same parent`
 - Similar methods : `insertAfter`, `before`, `insertBefore`

- Attribute Methods like `css()`, `attr()`, `val()`, `html()`, `text()` can be used for both setting and getting attributes
- Based on whether one or two arguments were passed
- Example:
 - For Setting
`$("#p:last").css("color", "green");`
 - For Getting
`let pcolor = $("#p:last").css("color")`

- Most jQuery methods return jQuery object
- You can chain them together to perform multiple operations on the same elements

```
$("#deleted").addClass("red").fadeOut("slow");
```

- This will not work as val() returns a string

```
$(":button").val("Click Me").click(function(){...})
```



THANK YOU

Vinay Joshi

Department of Computer Science and Engineering

vinayj@pes.edu

+91 80 2672 6622



WEB TECHNOLOGIES

jQuery – Events and Effects

Vinay Joshi

Department of
Computer Science and Engineering

jQuery – Events and Effects

The Action Part

Action	Example
Events	click(), bind(), unbind(), live()
Effects	hide(), fadeOut(), toggle(), animate()

jQuery – Events

Event Methods

- Register an event handler for an element object passing a function reference to the event method

```
$(“span#message”).click(function(event){...})
```

; OR

```
$(“span#message”).on(“click”, function(event) {...});
```

- Like with attribute methods, the event methods have a different meaning without arguments
- Without the function reference argument, the event methods are treated like a manual firing of event

```
$("#span#message").click();
```


- Special event to register for handler once the document has been loaded completely is ready

```
$(document).ready(function(){  
    ...  
});
```

- within any event handler function **this** element refers to the element for which the handler is called

```
$( "p" ).click(function() {  
    var htmlString = $( this ).html();  
    ...  
}
```

- Lot of animation/styling effects can be accomplished using the effects methods like hide, show, toggle, fadeIn, fadeout etc.

```
$(“a#show-cart”).click(function(){  
    $(“#cart”).slideToggle(“slow”);  
});
```

- Or, use the animate method to build custom animations

```
$(“#showdown”).click(function(){  
    $(“#my-  
        div”).animate({  
        width: “70%”,  
        opacity: 0.4,  
        fontSize: “3em”  
    }, 1200 );  
});
```



THANK YOU

Vinay Joshi

Department of Computer Science and Engineering

vinayj@pes.edu

+91 80 2672 6622



WEB TECHNOLOGIES

jQuery AJAX and fetch() methods

Vinay Joshi

Department of
Computer Science and Engineering

- jQuery provides methods that use XMLHttpRequest internally to make AJAX requests
- The methods are
 - \$.ajax
 - \$.get
 - \$.post
 - \$.load
- In a single method call, the entire functionality of making an AJAX call using XMLHttpRequest and updating the page can be achieved

The **ajax()** method is used to perform an AJAX (asynchronous HTTP) request.

All jQuery AJAX methods use the **ajax()** method. This method is mostly used for requests where the other methods cannot be used.

Syntax

```
$.ajax({name:value, name:value, ... })
```

The parameters specifies one or more name/value pairs for the AJAX request.

jQuery AJAX methods

\$.ajax

- Performs an async AJAX request
- Syntax
 - \$.ajax({name:value, name:value, ... })
- Example
 - \$.ajax({url: "demo_test.txt", method: 'get', success: function(result){
 \$("#div1").html(result);
 }});
 - \$.ajax('/jquery/submitData', {
 type: 'POST', // http method
 data: { myData: 'This is my data.' }, // data to submit
 success: function (data, status, xhr) { // success callback function
 \$('p').append('status: ' + status + ', data: ' + data);
 },
 error: function (jqXHR, textStatus, errorMessage) {
 \$('p').append('Error' + errorMessage);
 }
});

The **\$.get()** method requests data from the server with an HTTP GET request.

Syntax:

```
$.get(URL,callback);
```

The required URL parameter specifies the URL you wish to request.

The optional callback parameter is the name of a function to be executed if the request succeeds.

jQuery AJAX methods

\$.get



- Syntax

- \$.get(url, [data],[callback]);

- Example

- \$.get('/jquery/getjsondata',
 {name:'Steve'},
 function (data, textStatus, jqXHR) {
 \$('p').append(data.firstName);
 }
);

// url

// request parameters

// success callback function

- Other Variants

- \$.getJSON(url, [data],[callback]);

- \$.getScript(url, [data],[callback]);

jQuery AJAX methods

\$.post

- Syntax
 - \$.post(url,[data],[callback],[type]);
- Example
 - ```
$.post('/jquery/submitJSONData', // url
 { myData: 'This is my data.' }, // data to be submitted
 function(data, status, jqXHR) { // success callback function
 $('p').append('status: ' + status + ', data: ' + data);
 },
 "json" //response type
);
```
  - Internally uses \$.ajax with method="post"

# jQuery AJAX methods

## \$.load

---

- Allows HTML or text content to be loaded from a server and added into an existing DOM element
- Syntax
  - `$( 'selector' ).load( url, [data], [callback], [type] );`
- Example
  - ```
$( '#msgDiv' ).load( 'getData', // url
    { name: 'bill' }, // data
    function( data, status, jqXHR ) { // success callback function
        console.log( 'data loaded' )
    }
);
```

fetch() AJAX method

Introduction

- Starts the process of fetching a resource from the network
- Returns a promise which is fulfilled once the response is available
- The promise resolves to the Response object representing the response to your request
- The promise does not reject on HTTP errors — it only rejects on network errors. You must use then handlers to check for HTTP errors.
- Syntax

```
const fetchResponsePromise = fetch(resource [, init])
```

fetch() AJAX method

Code Example

```
const mydiv = document.querySelector('.my-div');

fetch('resp.html')           // returns a promise that resolves on response object
.then(function(response) {
    return response.text(); // returns a promise that resolves on text response
})
.then(function(text) {
    mydiv.innerHTML= text;
});
```



THANK YOU

Vinay Joshi

Department of Computer Science and Engineering

vinayj@pes.edu

+91 80 2672 6622